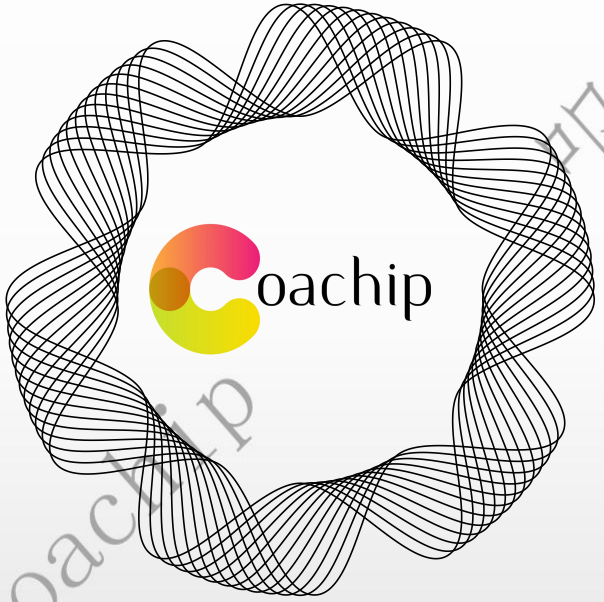


SV的interface

叩特电子Coachip

课程内容及目标

课程内容：

- 使用接口简化连接
- 连接接口和端口
- 接口的信号分组(modport)
- 创建接口监视模块
- 接口的优缺点



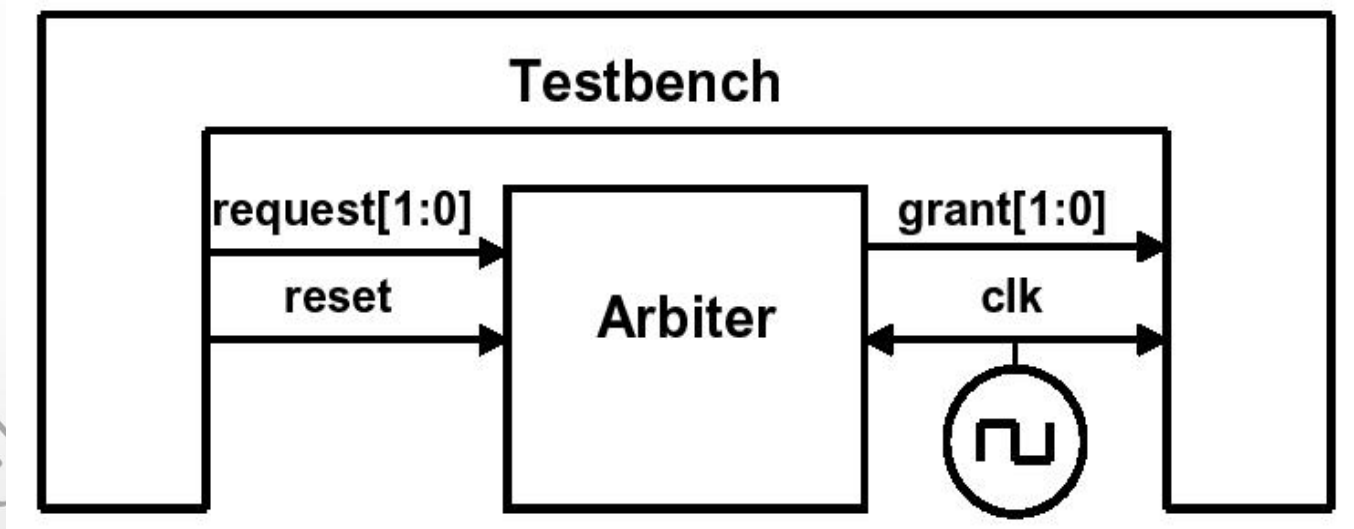
目标：

- 了解接口的使用

叩特电子Coachip

测试平台和DUT之间的通信

测试平台和一个仲裁器的连接实例



测试平台连接到 arbiter的例子, 包括测试平台, arbiter, 时钟发生器和连接的信号。

叩特电子Coachip

测试平台和一个仲裁器的连接实例

- 使用端口的仲裁器模型

```
module  arb_port(output logic [1:0] grant,
                 output logic       grant_valid,
                 input  logic [1:0] request,
                 input  logic       rst,
                 input  logic       clk);

  …
  always@(posedge clk or posedge rst)begin
        if(rst)
              grant <=2'b00;
        else
              …
  end
endmodule
```

叩特电子Coachip

测试平台和一个仲裁器的连接实例

- 使用端口的测试平台

```
module  test(input  logic [1:0] grant,
             input  logic       grant_valid,
             output logic [1:0] request,
             input  logic       rst,
             input  logic       clk);
  initial begin
       @(posedge clk) request <=2'b01;
       $display("@%05: Drove req=01",$time);
        repeat(2) @(posedge clk )
        if(grant_valid &&(grant!=2'b01))
             $display("@%0t:a1:grant!=2'b01",$time);
        …
        $finish;
  end
endmodule
```

叩特电子Coachip

测试平台和一个仲裁器的连接实例

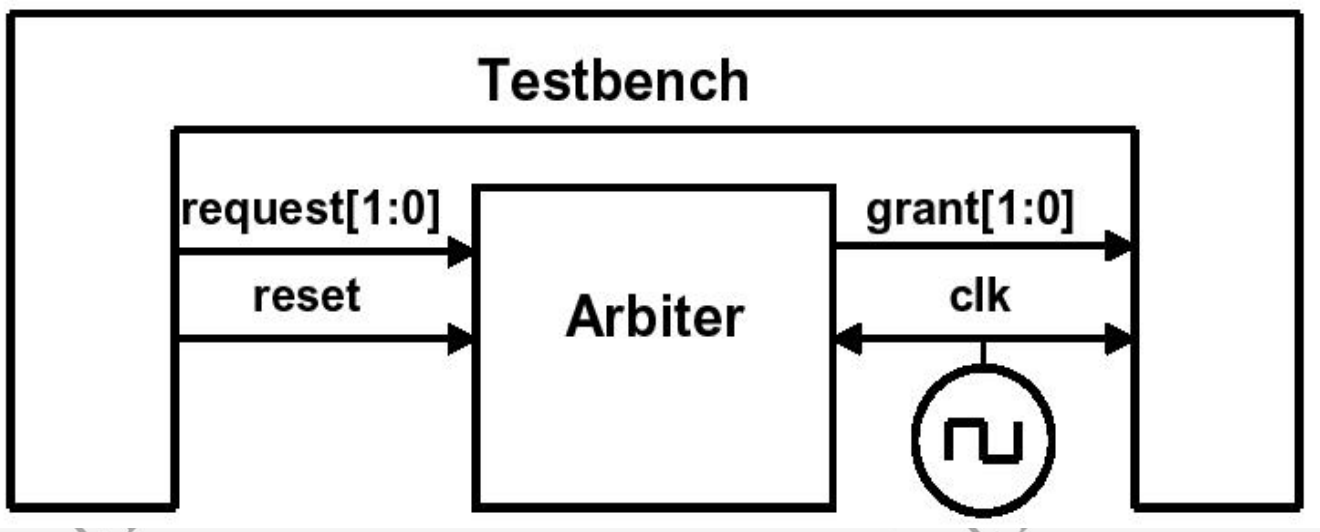
- 顶层连接

```
module  top;
    logic[1:0]grant, request;
    logic grant_valid;
    bit clk,rst;
    always #5 clk=~clk;

    arb_port a1(grant, grant_valid, request, rst, clk);
    test     t1(grant, grant_valid, request, rst, clk);
endmodule
```

叩特电子Coachip

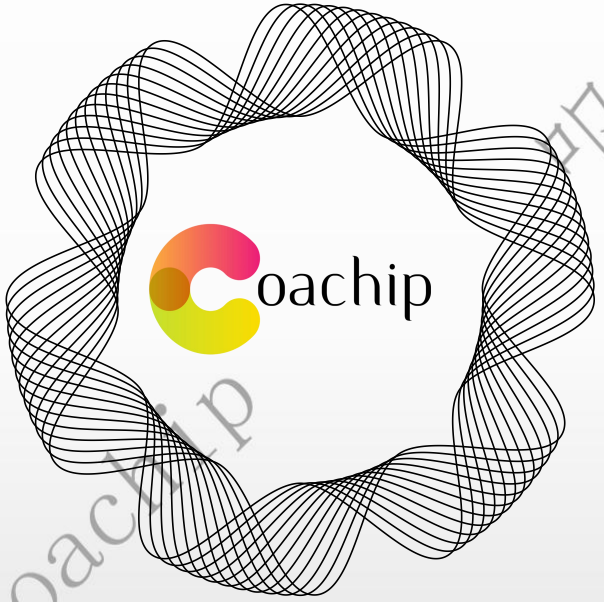
测试平台和一个仲裁器的连接实例



这种方法的缺点就是若DUT arb_port的端口需要改变，在test 和top中的声明都要改变。

在真实的设计中，往往含有数百个端口信号，因此，sv中推荐使用一种interface的结构。

叩特电子Coachip



SV的接口(interface)

叩特电子Coachip

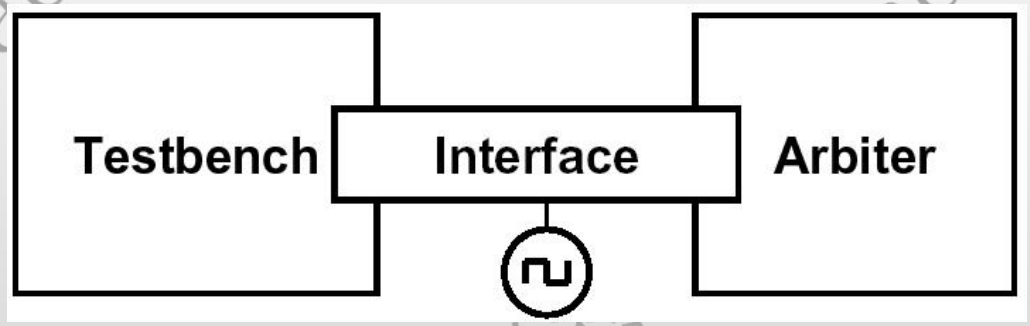
Systerm Verilog的接口(interface)

- 使用接口简化连接

  逻辑设计日益复杂，模块之间的通信必须分割成独立的实体。
  SV使用接口来连接不同模块。接口可以看作一捆智能连线.
  它包括了连接，同步;
  它连接了DUT和验证平台.

接口声明:

```
interface arb_if( input  bit clk);
        logic[1:0] grant,request;
        logic grant_valid;
        logic  rst;
endinterface
```



叩特电子Coachip

使用接口简化连接

- 使用接口的仲裁器:

```
module arb(arb_if  arbif);
    …
    always@(posedge arbif.clk or posedge arbif.rst);
      begin
          if(arbif.rst)
                        arbif.grant <=2'b00;
          else
                        arbif.grant <= next_grant;
    …
      end
endmodule
```

即特电子Coachip

使用接口简化连接

- 使用接口的测试平台:

```
module test(arb_if  arbif);
    …
     initial begin
            …
          @(posedge arbif.clk);
           arbif.request <= 2'b01;
           $display("@%0t: Drove req=01",$time);
           repeat(2) @(posedge arbif.clk);
           if(arbif.grant != 2'b01)
                 $display("@%0t: a1:frant!=2'b01",$time);
           $finish;
       end
endmodule:test
```

叩特电子Coachip

使用接口简化连接

采用接口连接:

```
module top;
    bit clk;
    always #5 clk=~clk;
    arb_if  arbif(clk);//例化接口
    arb a1(arbif);      //连接DUT
    test t1(arbif);     //连接tb
endmodule:top
```

采用端口连接:

```
module  top;
    logic[1:0]grant, request;
    bit clk,rst;
    always #5 clk=~clk;
    arb_port a1(grant, grant_valid, request,
                rst, clk);
    test t1(grant, grant_valid, request, rst,
                clk);
endmodule
```

叩特电子Coachip

连接接口和端口

若仲裁器的端口无法修改为接口,则需要连接验证人员自己定义的接口到端口上:

```
module top;
    bit clk;
    always #5 clk=~clk;
    arb_if  arbif(clk);                    //例化接口
    arb_port a1(.grant(arbif.grant), //连接DUT
                .grant_valid(arbif.grant_valid),
                .request(arbif.request),
                .rst (arbif.rst),
                .clk (arbif.clk));
     test t1(arbif);              //连接tb
endmodule:top
```

叩特电子Coachip

使用modport将接口中的信号分组

```
interface arb_if( input  bit clk);
        logic[1:0] grant,request;
        logic grant_valid;
        logic  rst;
endinterface
```

没有指定方向！！！

叩特电子Coachip

使用modport将接口中的信号分组

- 接口中使用modport结构将信号分组并指定方向

```
interface arb_if( input  bit clk);
        logic[1:0] grant,request;
        logic  rst;
        modport   TEST(output request, rst,
                              input  grant, clk);
        modport  DUT(input request, rst, clk,
                             output  grant);
        modport  MONITOR(input request, grant, rst, clk);
endinterface
```

叩特电子Coachip

使用modport将接口中的信号分组

- 则module定义的时候端口例化就变成：

```
module arb( arb_if.DUT arbif);
…
endmodule

module test(arb_if.TEST arbif);
…
endmodule

module monitor(arb_if.MONITOR arbif);
…
endmodule
```

叩特电子Coachip

创建接口监视模块

```
module monitor(arb_if.MONITOR arbif);
  always@(posedge arbif.request[0]) begin
         $display("@%0t:request[0] asserted",$time);
          @(posedge arbif.grant[0])
          $display("@%0t:grant[0] asserted",$time);
  end
  always@(posedge arbif.request[1]) begin
         $display("@%0t:request[1] asserted",$time);
          @(posedge arbif.grant[1])
          $display("@%0t:grant[1] asserted",$time);
  end
endmodule
```

叩特电子Coachip

接口interface的优缺点

- 优点:
  1) 便于设计重用;
  2) 可以替代原来需要在模块或者程序中重复声明并且位于代码内部的一系列信号,减少连接错误的可能性
  3) 要增加一个新信号时, 只需要在接口中声明一次, 不需要在更高层的模块层声明, 进一步减少错误;
  4) modport允许一个模块将接口的一系列信号捆绑到一起, 为信号指定方向;

- 缺点:
  1) 对于点对点的连接, 使用modport跟使用信号列表的端口一样冗余;
  2) 必须同时使用信号名和接口名,会使模块变得更加冗长;
  3) 如果连接两个模块使用的是一个不会被重用的专用协议,使用接口需要做比端口连线更多的工作;
  4) 连接两个不同的接口很困难;一个新的接口可能包含了现有接口的所有信号并新增了信号, 需要拆分独立的信号并正确的驱动;

即特电子Coachip

练习

根据design，设计出test bench，分别用端口连接、接口连接实现，并体会他们的异同

```
module  arb_port(output logic [1:0] grant,
                 output logic       grant_valid,
                 input  logic [1:0] request,
                 input  logic       rst,
                 input  logic       clk);
logic  pri;
always_ff @(posedge clk or negedge rst)
begin
   if(rst)
      pri <=0;
   else
      pri<=~pri;
end
always_comb begin
   grant_valid = |request;
   if(request== 2'b01 || reqeust==2'b10)
      grant = request;
   else if (request==2'b11)
      unique case(pri)
         1'b0: grant = 2'b01;
         1'b1: grant = 2'b10;
      endcase
   else begin
      grant_valid = 0;
      grant =0;
   end
end
endmodule
```

```
module  test(input  logic [1:0] grant,
             input  logic
grant_valid,
             output logic [1:0] request,
             input  logic       rst,
             input  logic       clk);
  initial begin
       @(posedge clk) request <=2'b01;
       $display("@%05: Drove
req=01",$time);
        repeat(2) @(posedge clk )
     ...
  end
endmodule
```

```
module test(arb_if  arbif);
 initial begin
       @(posedge arbif.clk) arbif.request
<=2'b01;
       $display("@%05: Drove
req=01",$time);
        repeat(2) @(posedge arbif.clk )
     ...
  end
endmodule
```

叩特电子Coachip

Q&A

昂特电子Coachip